Comprendre ton Makefile AVR

Le Makefile automatise les commandes qui transforment ton fichier C en programme pour le microcontrôleur, puis qui transfèrent ce programme sur la carte. Le cours est en français ; les commentaires du fichier `Makefile` sont en anglais, comme ceux de ton code.

1. Les trois grandes étapes

Étape	Outil	Entrée	Résultat
Compiler et assembler le programme final	`avr-gcc`	`main.c`	`main.bin`, qui contient ici un programme au format ELF
Convertir le format du fichier	`avr-objcopy`	`main.bin`	`main.hex`, au format Intel HEX
Programmer la carte	`avrdude`	`main.hex`	Le programme est écrit dans la mémoire flash du microcontrôleur

Le compilateur tourne sur ton Mac, mais les instructions qu'il produit sont destinées à l'ATmega328P : c'est une **compilation croisée**.

Attention au nom `main.bin` : l'extension ne détermine pas le contenu du fichier ; avec la commande de ton Makefile, AVR-GCC produit normalement un fichier ELF, même si tu lui donnes un nom terminé par `.bin`.

Une extension `.elf` rendrait le nom plus explicite ; j'ai conservé `.bin` pour rester proche de ton fichier.

2. Les mots techniques, expliqués en une phrase

Mot	Explication
AVR	AVR désigne l'architecture et la famille de microcontrôleurs dont fait partie l'ATmega328P.
Microcontrôleur / MCU	Un microcontrôleur est une puce qui réunit un processeur, de la mémoire et des périphériques pour piloter un système électronique.
CPU	Le CPU est la partie du microcontrôleur qui exécute les instructions du programme.
Périphérique	Un périphérique est un bloc matériel du microcontrôleur, comme un port GPIO, un timer ou un UART.
GCC	GCC signifie GNU Compiler Collection et désigne une collection d'outils de compilation.
Compilateur	Un compilateur traduit le code d'un langage, comme le C, vers un code destiné à la machine cible.
Assembleur	L'assembleur transforme des instructions écrites en langage assembleur en code machine.
Éditeur de liens / linker	L'éditeur de liens réunit les morceaux compilés et le code nécessaire au démarrage pour produire le programme final.
Préprocesseur	Le préprocesseur traite notamment les `#include`, les `#define` et les conditions `#if` avant la compilation du C.
Macro	Une macro du préprocesseur est un nom dont l'expansion fournit du texte ou une valeur lors du traitement du code source.
Horloge	L'horloge fournit des impulsions régulières qui cadencent le fonctionnement du processeur.
Hertz / Hz	Un hertz correspond à un cycle par seconde.
Mégahertz / MHz	Un mégahertz correspond à un million de cycles par seconde.
Bit	Un bit est une information binaire qui vaut `0` ou `1`.
Octet / byte	Un octet contient huit bits.
UART	L'UART est un périphérique matériel qui transmet et reçoit des données par une liaison série asynchrone.
Liaison série	Une liaison série transmet les bits successivement sur une ligne de communication.
Baud	Le baud mesure le nombre de symboles transmis par seconde et correspond ici au nombre de bits transmis par seconde.

Mot	Explication
Protocole	Un protocole définit les règles suivies par deux appareils pour échanger des informations.
Bootloader	Un bootloader est un petit programme déjà présent sur la puce qui peut recevoir une nouvelle application et l'installer en mémoire.
Flash	La mémoire flash conserve le programme même lorsque la carte n'est plus alimentée.
ELF	ELF, pour Executable and Linkable Format, est un format structuré contenant notamment le code compilé et des informations sur son organisation.
Intel HEX	Intel HEX est un format texte qui décrit des données à programmer, leurs adresses et des sommes de contrôle.
Somme de contrôle / checksum	Une somme de contrôle est une valeur calculée à partir de données pour aider à détecter des erreurs.
Symbole	Un symbole associe un nom, comme celui d'une fonction, à un élément du programme compilé.
Firmware	Un firmware est le programme destiné à fonctionner dans un appareil électronique, ici ton microcontrôleur.

3. Les variables du Makefile

Une ligne comme `MCU = atmega328p` stocke du texte dans une variable de **make**. `$(MCU)` insère ensuite ce texte dans une règle ou une commande.

```
MCU = atmega328p
```

Donc `-mmcu=$(MCU)` devient `-mmcu=atmega328p`.

Ces variables sont différentes des variables de ton programme C : elles servent à construire les commandes sur ton ordinateur.

Variable	Valeur de ton fichier	Rôle
`CC`	`avr-gcc`	Choisit le compilateur pour les AVR.
`MCU`	`atmega328p`	Indique le modèle de puce ciblé.
`CPU_SPEED`	`16000000UL`	Fournit la fréquence attendue au code C par la macro `F_CPU`.
`AVR_DUDE`	`avrdude`	Choisit l'outil de programmation de la carte.
`BOARD`	`arduino`	Choisit la méthode de communication avec le bootloader dans AVRDUDE.

Variable	Valeur de ton fichier	Rôle
`OBJ_CPY`	`avr-objcopy`	Choisit l'outil de conversion du fichier compilé.
`BAUD_RATE`	`115200`	Choisit le débit de la liaison série utilisée pour le transfert.
`PORT`	`/dev/ttyUSB0`	Indique le chemin du périphérique série dans le système d'exploitation.
`CFLAGS`	Plusieurs options	Regroupe les options transmises à AVR-GCC.
`SRCS`	`main`	Fournit ici un seul nom de base : `main.c`, `main.bin` et `main.hex`.
`RM`	`rm -rf`	Fournit la commande utilisée pour supprimer les fichiers générés.

`CC` et `CFLAGS` sont des noms conventionnels ; beaucoup d'autres noms sont choisis par l'auteur du Makefile. Ici, `SRCS` ne contient pas une liste de fichiers sources : il contient seulement `main`. La règle de compilation actuelle traite donc un seul fichier C.

Fréquence du CPU : `16000000UL`

`16000000` signifie 16 millions ; `UL` indique que la constante C est de type entier non signé `unsigned long`. La fréquence est ici de **16 MHz**, donc 16 millions de cycles d'horloge par seconde.

Un cycle n'est pas obligatoirement une instruction complète : certaines instructions peuvent prendre plusieurs cycles.

La commande `-DF_CPU=16000000UL` rend cette valeur disponible avant le traitement de `util/delay.h`, qui l'utilise pour calculer les délais.

Définir `F_CPU` ne règle pas matériellement la fréquence du microcontrôleur. La valeur doit correspondre à son horloge réelle ; sinon, les durées calculées peuvent être incorrectes.

Débit : `115200`

Pour cette liaison série, `115200` correspond à 115 200 bits transmis par seconde. Cela ne signifie pas 115 200 octets par seconde : un octet contient huit bits, et la transmission ajoute des bits de cadrage.

Par exemple, en configuration série 8N1, un octet de données utilise dix bits transmis : un bit de départ, huit bits de données et un bit d'arrêt. Le plafond théorique de données est alors 11 520 octets par seconde, avant les échanges du protocole.

La fréquence CPU et le débit série sont deux réglages différents : l'une décrit la vitesse de l'horloge du processeur, l'autre la vitesse de communication.

Chemin de la carte sur macOS

`/dev/ttyUSB0` est un exemple Linux. Sur ton Mac, le nom est généralement de la forme `/dev/cu.usbmodem...` ou `/dev/cu.usbserial...`, selon la carte et son interface USB-série.

Pour regarder les périphériques présents sur **ton Mac** :

```
ls /dev/cu.*
```

Tu peux ensuite remplacer la valeur dans le Makefile ou la fournir à make :

```
make flash PORT=/dev/cu.usbmodemXXXX
```

XXXX est un exemple à remplacer par le vrai nom, pas une valeur à copier telle quelle. La carte doit être connectée pour le transfert.

4. Les options de compilation

Ta ligne CFLAGS devient :

```
avr-gcc -Os -Wall -Wextra -Werror -mmcu=atmega328p -DF_CPU=16000000UL \
    -DUART_BAUDRATE=115200 -o main.bin main.c
```

Option	Explication
`-Os`	Demande des optimisations visant à réduire la taille du programme.
`-Wall`	Active un ensemble important d'avertissements, mais pas tous les avertissements disponibles.
`-Wextra`	Active des avertissements supplémentaires.
`-Werror`	Transforme les avertissements activés en erreurs qui empêchent la réussite de la compilation.
`-mmcu=atmega328p`	Sélectionne le modèle AVR pour la compilation et l'édition de liens ; les définitions choisies permettent aussi à `avr/io.h` de charger les bons registres.
`-DF_CPU=16000000UL`	Définit la macro C F_CPU, utilisée notamment pour calculer les délais.
`-DUART_BAUDRATE=115200`	Définit une macro personnalisée appelée `UART_BAUDRATE` avec la valeur `115200`.
`-o main.bin`	Choisit le nom du fichier produit par la commande.
`main.c`	Indique le fichier source à traiter.

Le `-D` veut dire « définir une macro ». Pour un objet simple comme F_CPU, c'est comparable à placer `#define F_CPU 16000000UL` avant les inclusions du programme.

`UART_BAUDRATE` **ne configure pas automatiquement l'UART.** Si ton code C ne lit jamais cette macro, elle ne change pas son comportement ; le `-b` de la commande AVRDUDE, lui, choisit réellement le débit de transfert.

Dans ton commentaire initial, tu avais `# -s ->` : **il n'y a pas d'option `-s` dans ta commande**. Si elle était ajoutée à la commande d'édition de liens, elle demanderait de retirer les informations de symboles. `-s`, `-Os` et `-o` sont trois options différentes ; la casse des lettres compte.

5. Comment lire une règle make

Une règle contient une cible, ses prérequis et éventuellement une recette :

```
cible: prerequis
    commande
```

La ligne de commande doit commencer par une **vraie tabulation**. Le fichier fourni utilise ces tabulations.

Élément	Signification
Cible	Le fichier à produire ou l'action demandée.
Prérequis	Les fichiers ou autres cibles qui doivent être prêts avant cette cible.
Recette	Les commandes que make exécute pour produire le résultat.
`:`	Sépare le nom de la cible de ses prérequis.
`#`	Commence un commentaire hors de la recette.
`$(NOM)`	Insère la valeur d'une variable de make.
`$@`	Représente le nom de la cible de la règle courante.
`$<`	Représente le premier prérequis de la règle courante.

Par exemple :

```
main.bin: main.c
    avr-gcc $(CFLAGS) -o $@ $<
```

Ici, `$@` devient `main.bin` et `$<` devient `main.c`.

Pour une cible qui est un fichier, make reconstruit normalement le résultat s'il manque ou si un prérequis est plus récent. Il ne recompilera donc pas forcément à chaque commande.

Cette version simple ne suit pas automatiquement les changements des fichiers `.h`, ni ceux des options de compilation : après un changement de ce genre, tu peux lancer `make clean` puis `make main.hex` pour reconstruire sans transfert.

6. La conversion en Intel HEX

```
avr-objcopy -O ihex main.bin main.hex
```

Élément	Explication
`avr-objcopy`	Lit le fichier compilé et produit un autre format.
`-O`	Choisit le format de sortie : lettre O majuscule.
`ihex`	Demande le format Intel HEX.

Élément	Explication
`main.bin`	Fichier d'entrée, ici au format ELF.
`main.hex`	Fichier de sortie contenant les données à programmer et leurs adresses.

La conversion ne recompile pas ton code C : elle change la représentation du programme déjà compilé.

7. La commande de transfert, morceau par morceau

```
avrdude -p atmega328p -c arduino -P /dev/ttyUSB0 -b 115200 -U flash:w:main.hex:i
```

Le chemin de port ci-dessus est celui de ton exemple ; sur macOS, remplace-le par celui de ta carte.

Option	Explication
`-p atmega328p`	Sélectionne le modèle de microcontrôleur à programmer.
`-c arduino`	Sélectionne le mode de programmation utilisé pour communiquer avec le bootloader Arduino.
`-P ...`	Sélectionne le chemin de la connexion série, avec P majuscule.
`-b 115200`	Sélectionne le débit de cette communication série.
`-U ...`	Demande une opération sur une mémoire du microcontrôleur.

Ne confonds pas `-p` et `-P` : le premier choisit la **puce**, le second la **connexion**.

La structure après `-U` est : `memoire:operation:fichier:format`.

Morceau de `flash:w:main.hex:i`	Signification
`flash`	Mémoire qui contient le programme.
`w`	Opération d'écriture, pour *write*.
`main.hex`	Fichier dont les données seront écrites.
`i`	Format Intel HEX du fichier d'entrée.

La commande demande donc : **« Écrire le programme décrit dans main.hex dans la mémoire flash. »**

Le bootloader est un programme sur la carte ; AVRDUDE est un programme sur ton ordinateur. Ils communiquent pour installer ton application.

8. Les cibles disponibles

Commande	Effet
make ou make all	Prépare main.hex, puis tente le transfert sur la carte.
make main.bin	Compile et assemble le programme final, sans le convertir ni le transférer.
make main.hex	Compile si nécessaire et prépare le HEX, sans transfert.
make flash	Prépare le HEX si nécessaire, puis tente le transfert.
make clean	Supprime les fichiers .bin et .hex générés dans le dossier courant.
make fclean	Fait la même chose que clean dans ce petit Makefile.
make re	Avec make ordinaire, nettoie puis relance la construction et le transfert.
make -n main.hex	Affiche les commandes prévues pour le HEX sans les exécuter.

all est la cible par défaut parce que c'est la première cible normale de ce fichier.

flash dépend lui-même du fichier HEX : make vérifie donc que celui-ci est prêt avant de lancer AVRDUDE.

Avec re: fclean all, les prérequis ne garantissent pas cet ordre sous make -j. Pour cet exercice et cette version conservée, utilise make re sans compilation parallèle.

Nettoyage

rm -rf *.bin *.hex supprime les fichiers qui correspondent à ces motifs dans le dossier courant. * est un joker qui correspond à une partie quelconque du nom ; -r autorise aussi la suppression récursive des dossiers ; -f supprime les demandes de confirmation et ignore les fichiers absents.

Pour les fichiers générés ici, rm -f suffirait : j'ai conservé ta commande originale.

.PHONY

.PHONY: all clean fclean flash re indique que ces noms désignent des actions, pas des fichiers à considérer comme déjà à jour.

Par exemple, même si un fichier appelé clean existe dans ton dossier, make clean exécutera la recette de nettoyage.

Les vrais fichiers main.bin et main.hex ne sont pas déclarés .PHONY, pour que make puisse décider s'il faut les reconstruire.

9. Ce qui a été ajusté dans le fichier fourni

• Tous les commentaires ont été remplis en anglais et les descriptions imprécises ont été corrigées.

• La conversion utilise `$@` à la place de `$(SRCS).hex` pour désigner sa sortie : dans cette règle, les deux donnent exactement `main.hex`.

• `re` a été ajouté à `.PHONY`.

• Les noms, les valeurs, les options de transfert et les autres règles sont conservés.

Le fichier peut être utilisé tel quel pour préparer le HEX si `main.c` est présent et que les outils AVR sont installés. Le transfert nécessite aussi une carte connectée et la bonne valeur de `PORT`.

10. Documentation officielle

• GNU make : structure d'une règle

• GNU make : variables automatiques

• GNU make : cibles PHONY

• GCC 9.5 : options AVR

• GCC 9.5 : options générales

• GCC 9.5 : avertissements

• GCC 9.5 : optimisation

• GCC 9.5 : préprocesseur

• AVR-LibC : délais et F_CPU

• GNU Binutils : objcopy

• AVRDUDE : options

Vérification de cette version

La syntaxe et l'expansion des commandes ont été vérifiées avec make en mode simulation, sur un dossier de test contenant un fichier `main.c`. Aucun transfert vers une carte n'a été effectué. Les outils AVR n'étant pas présents dans l'environnement de vérification, ce contrôle ne constitue pas une compilation réelle de ton programme.